

Experiment - 11

Student Name: Mimanshu Gahlaut

Branch: CSE - AIML

Semester: 5

Subject Name: ADBMS

UID: 24BAI70038

Section/Group: 24AIT_KRG-1_B

Date of Performance: 21/9/26

Subject Code: 24CSP-310

(A)

A company stores the monthly salary history of its employees in an EmployeeSalaryHistory table.

Write an SQL query to find the employees whose salary has increased in every recorded month compared to the immediately previous month. For each salary record, compare the current month's salary with the employee's previous month's salary using a Window Function. Display only those employees for whom all later salary records are greater than their respective previous salary.

The output should display:

Employee ID

Employee Name

Output:

employee_id	employee_name
101	Alok
103	Raj

Solution:

```
CREATE TABLE EmployeeSalaryHistory (  
    record_id INT PRIMARY KEY,  
    employee_id INT NOT NULL,  
    employee_name VARCHAR(50) NOT NULL,  
    salary_month DATE NOT NULL,  
    salary INT NOT NULL  
);
```

```
INSERT INTO EmployeeSalaryHistory
(record_id, employee_id, employee_name, salary_month, salary)
VALUES
(1, 101, 'Alok', '2026-01-01', 50000),
(2, 101, 'Alok', '2026-02-01', 52000),
(3, 101, 'Alok', '2026-03-01', 55000),

(4, 102, 'Priya', '2026-01-01', 60000),
(5, 102, 'Priya', '2026-02-01', 58000),
(6, 102, 'Priya', '2026-03-01', 62000),

(7, 103, 'Raj', '2026-01-01', 45000),
(8, 103, 'Raj', '2026-02-01', 47000),
(9, 103, 'Raj', '2026-03-01', 49000),

(10, 104, 'Sneha', '2026-01-01', 70000),
(11, 104, 'Sneha', '2026-02-01', 70000),
(12, 104, 'Sneha', '2026-03-01', 72000);
```

```
SELECT employee_id, employee_name
FROM (
    SELECT *,
           LAG(salary) OVER(
               PARTITION BY employee_id
               ORDER BY salary_month
           ) prev_salary
    FROM EmployeeSalaryHistory
) E
GROUP BY employee_id, employee_name
HAVING COUNT(*) FILTER (
    WHERE salary <= prev_salary
) = 0;
```

(B)

A company maintains employee details in an Employee table and department information in a Department table.

Write an SQL query to find the employees whose salaries belong to the top three distinct salary levels within their respective departments. If multiple employees have the same salary, they should receive the same salary rank and all of them should be included in the result.

The output should display:

Department Name

Employee Name
Salary

Output:

Department	Employee	Salary
IT	Max	90000
IT	Jim	85000
IT	John	85000
IT	Alex	75000
Sales	Henry	80000
Sales	Susan	75000
Sales	David	70000
HR	Tina	60000
HR	Rita	55000
HR	Manoj	50000

Solution:

```
CREATE TABLE Department (
    id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL
);
INSERT INTO Department (id, name)
VALUES
(1, 'IT'),
(2, 'Sales'),
(3, 'HR');
```

```
CREATE TABLE Employee (
    id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    salary INT NOT NULL,
    departmentId INT,
    FOREIGN KEY (departmentId)
        REFERENCES Department(id)
);
```

```
INSERT INTO Employee
(id, name, salary, departmentId)
VALUES
```

```
(1, 'Max', 90000, 1),
(2, 'Jim', 85000, 1),
(3, 'John', 85000, 1),
(4, 'Alex', 75000, 1),
(5, 'Sam', 70000, 1),

(6, 'Henry', 80000, 2),
(7, 'Susan', 75000, 2),
(8, 'David', 70000, 2),
(9, 'Robert', 65000, 2),

(10, 'Tina', 60000, 3),
(11, 'Rita', 55000, 3),
(12, 'Manoj', 50000, 3);
```

--Solution:

```
SELECT D.name AS Department,
       E.name AS Employee,
       E.salary
FROM (
    SELECT *,
           DENSE_RANK() OVER (
               PARTITION BY departmentId
               ORDER BY salary DESC
           ) AS rnk
    FROM Employee
) E
JOIN Department D
ON E.departmentId = D.id
WHERE rnk <= 3;
```